MANUAL TECNICO – PRACTICA 3:

Pablo Daniel Solares Samayoa

Carne: 202400044

Sección: C

ÍNDICE

1. Introducción 3
2. Arquitectura del Sistema 3
3. Estructura del Proyecto 3
4. Base de Datos 3
5. Endpoints Principales de la API 5
6. Funcionalidades Principales 5
6.1 Sistema de Autenticación 5
6.2 Gestión de Publicaciones 5
6.3 Sistema de Comentarios 6
6.4 Perfiles de Usuario 6
7. Componentes React Principales 8
Login.jsx 8
Register.jsx 9
Menu.jsx 10
CreatePubli.jsx 12
Comentarios.jsx 14
Perfil.jsx 16
8. Configuración y Despliegue 17
9. Seguridad 17

1. Introducción

Este documento describe la implementación técnica de una aplicación web para evaluación de cursos y catedráticos desarrollada para la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala. El sistema permite a los estudiantes compartir opiniones y experiencias sobre cursos y profesores mediante un sistema de publicaciones y comentarios.

2. Arquitectura del Sistema

El sistema sigue una arquitectura cliente-servidor con las siguientes tecnologías:

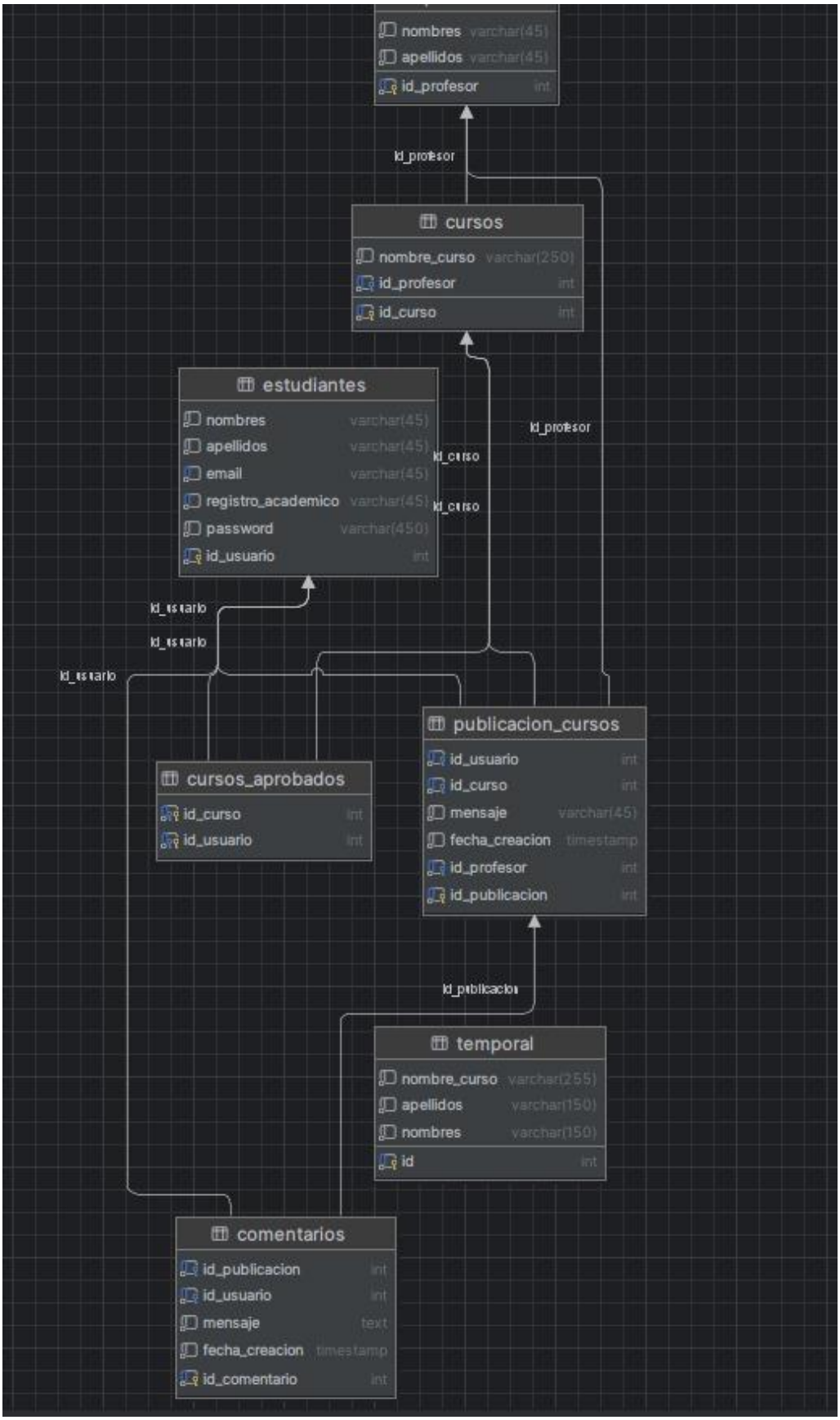
- **Frontend:** React.js con Bootstrap
- **Backend:** Node.js con Express.js
- **Base de Datos:** MySQL
- **Comunicación:** REST API

3. Estructura del Proyecto

- **Backend (Node.js + Express)**
- **Frontend (React.js)**

4. Base de Datos

El sistema utiliza MySQL con las siguientes tablas principales:


nombres varchar(45)
apellidos varchar(45)
id_profesor int
id_profesor
cursos
nombre_curso varchar(250)
id_profesor int
id_curso int
estudiantes
nombres varchar(45)
apellidos varchar(45)
email varchar(45)
registro_academico varchar(45)
password varchar(450)
id_usuario int
id_profesor
id_curso
id_curso
id_estudiante
id_estudiante
id_estudiante
publicacion_cursos
id_usuario int
id_curso int
mensaje varchar(45)
fecha_creacion timestamp
id_profesor int
id_publicacion int
cursos_aprobados
id_curso int
id_usuario int
id_publicacion
temporal
nombre_curso varchar(255)
apellidos varchar(150)
nombres varchar(150)
id int
comentarios
id_publicacion int
id_usuario int
mensaje text
fecha_creacion timestamp
id_comentario int

5. Endpoints Principales de la API

Autenticación

- POST /usuarios/register - Registrar nuevo usuario
- POST /usuarios/login - Iniciar sesión
- POST /usuarios/recuperarPass - Recuperar contraseña

Publicaciones

- POST /publicacion/crearPublicacion - Crear nueva publicación
- GET /publicacion/obtenerPubli - Obtener todas las publicaciones
- GET /publicacion/obtenerPubliID/:id - Obtener publicación específica
- GET /publicacion/obtenerPorCurso/:id_curso - Filtrar por curso
- GET /publicacion/obtenerPorProfe/:id_profesor - Filtrar por profesor

Comentarios

- POST /comentario/crear - Crear comentario
- GET /comentario/get/:id_publicacion - Obtener comentarios de publicación

Datos

- GET /cursos/get - Obtener listado de cursos
- GET /profesores/get - Obtener listado de profesores

6. Funcionalidades Principales

6.1 Sistema de Autenticación

- Registro de usuarios con validación de campos
- Inicio de sesión con verificación de credenciales
- Recuperación de contraseña con validación por correo y registro académico
- Almacenamiento seguro de contraseñas con bcrypt

6.2 Gestión de Publicaciones

- Creación de publicaciones sobre cursos o catedráticos
- Sistema de filtros por curso, catedrático y estudiante
- Visualización ordenada por fecha (más recientes primero)

- Integración con datos maestros de cursos y profesores

6.3 Sistema de Comentarios

- Comentarios en publicaciones existentes
- Visualización de comentarios con información de usuario
- Interfaz responsive para agregar y ver comentarios

6.4 Perfiles de Usuario

- Visualización de información personal
- Edición de datos de perfil (excepto registro académico)
- Navegación entre diferentes secciones del sistema



7. Componentes React Principales

Login.jsx

Componente de inicio de sesión con validación de credenciales y manejo de estados de error.



```jsx
import React, { useState } from "react";
import { Link, useNavigate } from "react-router-dom";
import axios from 'axios';
export default function Login() {
    const[reg, setReg] = useState('');
    const[password, setPassword] = useState('');
    const[errorMsg, seterrorMsg] = useState('');
    const[successMsg, setsuccessMsg] = useState('');

    const navigate = useNavigate();
    const handleSubmit  =  (e) => {
       e.preventDefault();
        seterrorMsg("");
        setsuccessMsg("");


       (async () => {
            try {
                const response = await axios.post('http://localhost:4000/usuarios/login', {
                    registro_academico: reg,
                    password: password
                });

                if(response.status === 200){
                    localStorage.clear();
                    localStorage.setItem("usuario", response.data.token.id_usuario);
                    setsuccessMsg(response.data.message || "Login exitoso ");
     navigate('/menu');

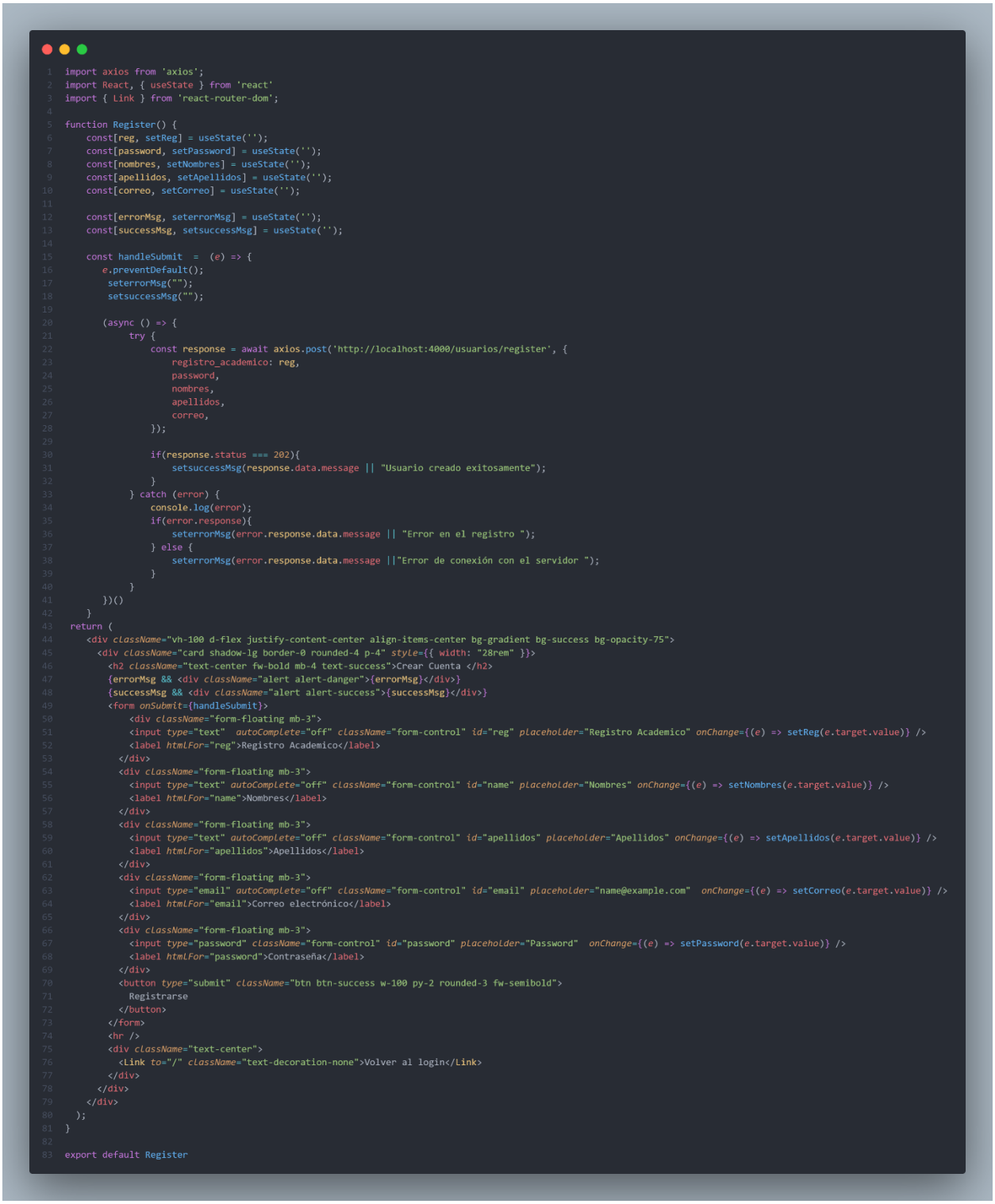
                }
            } catch (error) {
                if(error.response){
                    seterrorMsg(error.response.data.error || "Error en el login ");
                } else {
                    seterrorMsg("Error de conexión con el servidor ");
                }
            }
       })()
    }


    return (
        <div className="vh-100 d-flex justify-content-center align-items-center bg-gradient bg-primary bg-opacity-75">
        <div className="card shadow-lg border-0 rounded-4 p-4" style={{ width: "25rem" }}>
            <h2 className="text-center fw-bold mb-4 text-primary">Bienvenido </h2>
            {errorMsg && <div className="alert alert-danger">{errorMsg}</div>}
        {successMsg && <div className="alert alert-success">{successMsg}</div>}
            <form onSubmit={handleSubmit}>
            <div className="form-floating mb-3">
                <input type="text" className="form-control" id="registro" placeholder="Registro Academico" autoComplete="off" onChange={(e) => setReg(e.target.value)} />
                <label htmlFor="registro">Registro Academico</label>
            </div>
            <div className="form-floating mb-3">
                <input type="password" className="form-control" id="password" placeholder="Password" onChange={(e) => setPassword(e.target.value)}/>
                <label htmlFor="password">Contraseña</label>
            </div>
            <button type="submit" className="btn btn-primary w-100 py-2 rounded-3 fw-semibold">
                Iniciar Sesión
            </button>
            </form>
            <hr />
            <div className="d-flex justify-content-between mt-2">
            <Link to="/register" className="text-decoration-none"> Registrarse</Link>
            <Link to="/recover" className="text-decoration-none"> ¿Olvidaste tu contraseña?</Link>
            </div>
        </div>
        </div>
    );
}

```

Register.jsx

Formulario de registro con validación de todos los campos requeridos.



```jsx
import axios from 'axios';
import React, { useState } from 'react'
import { Link } from 'react-router-dom';

function Register() {
    const[reg, setReg] = useState('');
    const[password, setPassword] = useState('');
    const[nombres, setNombres] = useState('');
    const[apellidos, setApellidos] = useState('');
    const[correo, setCorreo] = useState('');

    const[errorMsg, seterrorMsg] = useState('');
    const[successMsg, setsuccessMsg] = useState('');

    const handleSubmit  =  (e) => {
       e.preventDefault();
        seterrorMsg("");
        setsuccessMsg("");

       (async () => {
            try {
                const response = await axios.post('http://localhost:4000/usuarios/register', {
                    registro_academico: reg,
                    password,
                    nombres,
                    apellidos,
                    correo,
                });

                if(response.status === 202){
                    setsuccessMsg(response.data.message || "Usuario creado exitosamente");
                }
            } catch (error) {
                console.log(error);
                if(error.response){
                    seterrorMsg(error.response.data.message || "Error en el registro ");
                } else {
                    seterrorMsg(error.response.data.message ||"Error de conexión con el servidor ");
                }
            }
        })()
    }
 return (
    <div className="vh-100 d-flex justify-content-center align-items-center bg-gradient bg-success bg-opacity-75">
      <div className="card shadow-lg border-0 rounded-4 p-4" style={{ width: "28rem" }}>
        <h2 className="text-center fw-bold mb-4 text-success">Crear Cuenta </h2>
        {errorMsg && <div className="alert alert-danger">{errorMsg}</div>}
        {successMsg && <div className="alert alert-success">{successMsg}</div>}
        <form onSubmit={handleSubmit}>
            <div className="form-floating mb-3">
            <input type="text"  autoComplete="off" className="form-control" id="reg" placeholder="Registro Academico" onChange={(e) => setReg(e.target.value)} />
            <label htmlFor="reg">Registro Academico</label>
          </div>
          <div className="form-floating mb-3">
            <input type="text" autoComplete="off" className="form-control" id="name" placeholder="Nombres" onChange={(e) => setNombres(e.target.value)} />
            <label htmlFor="name">Nombres</label>
          </div>
          <div className="form-floating mb-3">
            <input type="text" autoComplete="off" className="form-control" id="apellidos" placeholder="Apellidos" onChange={(e) => setApellidos(e.target.value)} />
            <label htmlFor="apellidos">Apellidos</label>
          </div>
          <div className="form-floating mb-3">
            <input type="email" autoComplete="off" className="form-control" id="email" placeholder="name@example.com"  onChange={(e) => setCorreo(e.target.value)} />
            <label htmlFor="email">Correo electrónico</label>
          </div>
          <div className="form-floating mb-3">
            <input type="password" className="form-control" id="password" placeholder="Password"  onChange={(e) => setPassword(e.target.value)} />
            <label htmlFor="password">Contraseña</label>
          </div>
          <button type="submit" className="btn btn-success w-100 py-2 rounded-3 fw-semibold">
            Registrarse
          </button>
        </form>
        <hr />
        <div className="text-center">
          <Link to="/" className="text-decoration-none">Volver al login</Link>
        </div>
      </div>
    </div>
  );
}

export default Register
```

Menu.jsx

Dashboard principal que muestra las publicaciones con sistema de filtros.

```jsx
import axios from "axios";
import React, { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";



export default  function  Menu() {
  const [filterCurso, setFilterCurso] = useState("");
const [filterProfesor, setFilterProfesor] = useState("");
const [filterEstudiante, setFilterEstudiante] = useState("");
  const [publicaciones, setPublicaciones] = useState([]);
    const id_usuario = localStorage.getItem("usuario");
  useEffect(() => {

    const llenarPubli = async () => {
        try {
        const response = await axios.get("http://localhost:4000/publicacion/obtenerPubli");
            if (response.status === 200) {

                setPublicaciones(response.data);
            }
        } catch (error) {
            console.error("Error al obtener publicaciones:", error);
        }
    }
    llenarPubli();
  }, []);

const publicacionesFiltradas = publicaciones.filter(pub =>
  (pub.nombre_curso?.toLowerCase() || "").includes(filterCurso.toLowerCase()) &&
  (pub.profesor?.toLowerCase() || "").includes(filterProfesor.toLowerCase()) &&
  (pub.estudiante?.toLowerCase() || "").includes(filterEstudiante.toLowerCase())
);
    const navigate = useNavigate();
    const cerrarSesion = (e) => {
      localStorage.clear();
      navigate('/');
    }

  return (
    <div className="d-flex vh-100 bg-light">


      <div className="bg-dark text-white d-flex flex-column p-3" style={{ width: "220px" }}>
        <h4 className="mb-4">Menú</h4>
        <ul className="nav nav-pills flex-column mb-auto">
          <li className="nav-item mb-2">
            <Link to={`/perfil/${id_usuario}`} className="nav-link text-white">Mi Perfil</Link>
          </li>
          <li className="nav-item mb-2">
            <Link to="/createPubli" className="nav-link text-white">Crear Publicación</Link>
          </li>
          <li className="nav-item mb-2">
            <button className="btn btn-danger w-100" onClick={(e) => cerrarSesion(e)}>Cerrar Sesión</button>
          </li>
        </ul>
      </div>


      <div className="flex-grow-1 p-4 overflow-auto">
        <h2 className="mb-4">Publicaciones</h2>


        <div className="row g-2 mb-4">
          <div className="col-md-3">
            <input type="text" className="form-control" placeholder="Filtrar por Curso" onChange={(e) => setFilterCurso(e.target.value)} />
          </div>
          <div className="col-md-3">
            <input type="text" className="form-control" placeholder="Filtrar por Catedrático" onChange={(e) => setFilterProfesor(e.target.value)} />
          </div>
          <div className="col-md-3">
            <input type="text" className="form-control" placeholder="Buscar por nombre" onChange={(e) => setFilterEstudiante(e.target.value)} />
          </div>
        </div>


        <div className="list-group">
          {publicacionesFiltradas.map((pub) => (
              <div key={pub.id_publicacion} className="list-group-item mb-3 shadow-sm rounded">
                <h5>{pub.nombre_curso} {pub.profesor}</h5>
                <p className="mb-1">{pub.mensaje}</p>
                <small>{pub.estudiante} | {pub.fecha_creacion}</small>
                <div className="mt-2">
                  <Link to={`/comentar/${pub.id_publicacion}`} className="btn btn-sm btn-outline-success">Comentar</Link>
                </div>
              </div>
            ))}
        </div>
      </div>
    </div>
  );
}
```

CreatePubli.jsx

Interfaz para creación de publicaciones con selección de curso o catedrático.



```jsx
import axios from "axios";
import React, { useEffect, useState } from "react";
import { Link, useNavigate } from "react-router-dom";

export default function CreatePub1() {
    const [tipo, setTipo] = useState('Curso');
    const [id_relacionado, setID] = useState(0);

    const [mensaje, setMensaje] = useState("");


    const [cursos, setCursos] = useState([]);
    const [profes, setProfes] = useState([]);
    const [errorMsg, setErrorMsg] = useState('');
    const [successMsg, setSuccessMsg] = useState('');
    useEffect(() => {
        const llenarCursos = async () => {
            try {
                const response = await axios.get('http://localhost:4000/cursos/get');
                if (response.status === 201) {
                    setCursos(response.data);
                }
            } catch (error) {
                console.error("Error al obtener cursos:", error);
            }
        };
        llenarCursos();
    }, []);

    useEffect(() => {
        const llenarProfes = async () => {
            try {
                const response = await axios.get(
                  "http://localhost:4000/profesores/get"
                );
                if (response.status === 201) {
                  setProfes(response.data);
                }
            } catch (error) {
                console.error("Error al obtener profes:", error);
            }
        };
        llenarProfes();
    }, []);

    const id_usuario = localStorage.getItem("usuario");
    const handleSubmit = async (e) => {
        e.preventDefault();

        setErrorMsg("");
        setSuccessMsg("");

        (async () => {
            try {
                if (tipo === "Curso") {
                    const response = await axios.post('http://localhost:4000/publicacion/crearPublicacion', {
                        id_usuario,
                        mensaje,
                        id_curso: id_relacionado,
                    });

                    if (response.status === 201) {
                        setSuccessMsg(response.data.message || "Publicacion Creada Correctamente");
                    }
                } else {
                    const response = await axios.post('http://localhost:4000/publicacion/crearPublicacion', {
                        id_usuario,
                        mensaje,
                        id_profesor: id_relacionado,
                    });

                    if (response.status === 201) {
                        setSuccessMsg(response.data.message || "Publicacion Creada Correctamente");
                    }
                }
            } catch (error) {
                console.log(error);
                if (error.response) {
                    setErrorMsg(error.response.data.message || "Error en el crear publicacion ");
                } else {
                    setErrorMsg(error.response.data.message || "Error de conexión con el servidor ");
                }
            }
        })()

    }
    const navigate = useNavigate();
    const cerrarSesion = (e) => {
      localStorage.clear();
      navigate("/");
    }
    return (
        <div className="d-flex vh-100 bg-light">
            <div
                 <nav
                 <ul
                    <li
                        <Link
                    </li>
                    <li
                        <Link to="/menu"
                            Ver Publicaciones
                        </Link>
                    </li>
                    <li
                        <button
                    </li>
                </ul>
            </div>

            <div
                <div
                    <h3
                    {errorMsg &&
    {successMsg &&
                    <form
                        <div
                            <label
                            <select
                                value={tipo}
                                onChange={(e) => setTipo(e.target.value)}
                            >
                                <option value="Curso">Curso</option>
                                <option value="CatedraLice">CatedraLice</option>
                            </select>
                        </div>
                        <div
                            <label
                            {tipo === "Curso" ? (
                                cursos && cursos.length > 0 ? (
                                    <select
                                        value={id_relacionado}
                                        onChange={(e) => setID(e.target.value)}
                                    >
                                        <option value="">Selecciona un curso</option>
                                        {cursos.map((c) => (
                                            <option key={c.id_curso} value={c.id_curso}>
                                                {c.nombre_curso}
                                            </option>
                                        ))}
                                    </select>
                                ) : (
                                    <p
                                )
                            ) : profes && profes.length > 0 ? (
                                <select
                                    value={id_relacionado}
                                    onChange={(e) => setID(e.target.value)}
                                >
                                    <option value="">Selecciona un profesor</option>
                                    {profes.map((p) => (
                                        <option key={p.id_profesor} value={p.id_profesor}>
                                            {p.nombres} {p.apellidos}
                                        </option>
                                    ))}
                                </select>
                            ) : (
                                <p
                            )}
                        </div>
                        <div
                            <label
                            <textarea
                        </div>
                        <button type="submit"
                            Publicar
                        </button>
                    </form>
                </div>
            </div>
        </div>
    );
}
```

Comentarios.jsx

Sistema de comentarios para publicaciones específicas.

```jsx
import React, { usefffect, useState } from "react";
import { useParams, Link } from "react-router-dom";
import axios from "axios";

export default function Comentarios() {
    const { id } = useParams();
    const [publicacion, setPublicacion] = useState(null);
    const [comentarios, setComentarios] = useState([]);
    const [nuevoComentario, setNuevoComentario] = useState("");
    const [errorMsg, setErrorMsg] = useState("");
    const [successMsg, setSuccessMsg] = useState("");


    useEffect(() => {
        const fetchData = async () => {
            try {
                const publiRes = await axios.get(
                    `http://localhost:4000/publicacion/obtenerPubliID/${id}`
                );
                if (publiRes.status === 200) {
                    setPublicacion(publiRes.data);
                }

                const comRes = await axios.get(
                    `http://localhost:4000/comentario/get/${id}`
                );
                if (comRes.status === 200) {
                    setComentarios(comRes.data);
                }
            } catch (error) {
                console.error("Error al cargar datos:", error);
            }
        };

        fetchData();
    }, [id]);


    const handleSubmit = async (e) => {
        e.preventDefault();
        setErrorMsg("");
        setSuccessMsg("");

        try {
            const id_usuario = localStorage.getItem("usuario");
            const response = await axios.post(
                `http://localhost:4000/comentario/crear`,
                {
                    id_usuario,
                    id_publicacion: id,
                    mensaje: nuevoComentario,
                }
            );

            if (response.status === 201) {
                setSuccessMsg("Comentario agregado correctamente");

            }
        } catch (error) {
            console.error(error);
            setErrorMsg("No se pudo enviar el comentario ");
        }
    };

    return (
        <div className="container my-5">
            <Link to="/menu" className="btn btn-secondary mb-3">
                ← Volver
            </Link>

            {publicacion ? (
                <div className="card shadow-sm p-4 mb-4">
                    <h4>
                        {publicacion.nombre_curso} - {publicacion.profesor}
                    </h4>
                    <p>{publicacion.mensaje}</p>
                    <small className="text-muted">
                        {publicacion.estudiante} | {publicacion.fecha_creacion}
                    </small>
                </div>
            ) : (
                <p>Cargando publicación...</p>
            )}

            <div className="card shadow-sm p-4">
                <h5>Comentarios</h5>
                <ul className="list-group mb-3">
                    {comentarios  ? (
                        comentarios.map((c) => (
                            <li
                                key={c.id_comentario}
                                className="list-group-item d-flex justify-content-between"
                            >
                                <span>Comentario: {c.mensaje}</span>
                                <small className="text-muted">
                                    Usuario: {c.nombres} {c.apellidos}  Fecha: {c.fecha_creacion}
                                </small>
                            </li>
                        ))
                    ) : (
                        <li className="list-group-item">No hay comentarios aún</li>
                    )}
                </ul>

                <form onSubmit={handleSubmit}>
                    <div className="mb-3">
                        <textarea
                            className="form-control"
                            rows="2"
                            placeholder="Escribe un comentario..."
                            value={nuevoComentario}
                            onChange={(e) => setNuevoComentario(e.target.value)}
                        />
                    </div>
                    <button type="submit" className="btn btn-primary w-100">
                        Comentar
                    </button>
                </form>

                {errorMsg && <p className="text-danger mt-2">{errorMsg}</p>}
                {successMsg && <p className="text-success mt-2">{successMsg}</p>}
            </div>
        </div>
    );
}

```

Perfil.jsx

Gestión y visualización de perfiles de usuario.



```jsx
import React, { useEffect, useState } from "react";
import axios from "axios";
import { Link } from "react-router-dom";

export default function Perfil() {
    const [usuario, setUsuario] = useState(null);
    const [nombres, setNombres] = useState("");
    const [apellidos, setApellidos] = useState("");
    const [email, setEmail] = useState("");
    const [successMsg, setSuccessMsg] = useState("");
    const [errorMsg, setErrorMsg] = useState("");

    const id_usuario = localStorage.getItem("usuario");


    useEffect(() => {
        const obtenerUsuario = async () => {
            try {
                const res = await axios.get(
                    `http://localhost:4000/usuarios/perfil/${id_usuario}`
                );
                if (res.status === 202) {
                    setUsuario(res.data);
                    setNombres(res.data.nombres);
                    setApellidos(res.data.apellidos);
                    setEmail(res.data.email);
                }
            } catch (error) {
                console.error("Error al obtener el usuario:", error);
                setErrorMsg("No se pudieron cargar los datos");
            }
        };
        obtenerUsuario();
    }, [id_usuario]);

    const handleSubmit = async (e) => {
        e.preventDefault();
        setSuccessMsg("");
        setErrorMsg("");

        try {
            const res = await axios.post(
                `http://localhost:4000/usuarios/actualizar/${id_usuario}`,
                { nombres, apellidos, email }
            );
            if (res.status === 202) {
                setSuccessMsg("Datos actualizados correctamente ");
            }
        } catch (error) {
            console.error(error);
            setErrorMsg("No se pudo actualizar la información ");
        }
    };

    if (!usuario) {
        return <div className="container mt-5">Cargando perfil...</div>;
    }

    return (
        <div class="d-flex vh-100 bg light">

            <div
                class="bg-dark text-white d-flex flex-column p-3"
                style={{ width: "220px" }}
            >
                <h4 class="mb-4">Menú</h4>
                <ul class="nav nav-pills flex-column mb-auto">
                    <li class="nav-item mb-2">
                        <link to="/menu" class="nav-link text-white">
                            Ver Publicaciones
                        </Link>
                    </li>
                    <li class="nav-item mb-2">
                        <Link to="/createPubli" class="nav-link text-white">
                            Crear Publicación
                        </Link>
                    </li>
                </ul>
            </div>


            <div class="container my-5">
                <div class="card shadow p-4">
                    <h3 class="mb-4"> Mi Perfil</h3>

                    {errorMsg && <div class="alert alert-danger">{errorMsg}</div>}
                    {successMsg && <div class="alert alert-success">{successMsg}</div>}

                    <form onSubmit={handleSubmit}>
                        <div class="mb-3">
                            <label class="form-label">Nombres</label>
                            <input
                                type="text"
                                class="form-control"
                                value={nombres}
                                onChange={(e) => setNombres(e.target.value)}
                            />
                        </div>
                        <div class="mb-3">
                            <label class="form-label">Apellidos</label>
                            <input
                                type="text"
                                class="form-control"
                                value={apellidos}
                                onChange={(e) => setApellidos(e.target.value)}
                            />
                        </div>
                        <div class="mb-3">
                            <label class="form-label">Correo electrónico</label>
                            <input
                                type="email"
                                class="form-control"
                                value={email}
                                onChange={(e) => setEmail(e.target.value)}
                            />
                        </div>
                        <button type="submit" class="btn btn primary w 100">
                            Actualizar
                        </button>
                    </form>
                </div>
            </div>
        </div>
    );
}
```

8. Configuración y Despliegue

Requisitos del Sistema

- Node.js 16+
- MySQL 5.7+
- Navegador web moderno

9. Seguridad

- Validación de entrada en todos los endpoints
- Hash de contraseñas con bcrypt (salt rounds: 10)
- Protección contra inyección SQL con consultas parametrizadas
- Validación de formato de correo electrónico
- Manejo centralizado de errores